A piecewise-linear function as a deep, narrow ReLU network

One kink per layer: the formula of every neuron

A continuous piecewise-linear function f with k kinks is a line plus k ReLU hinges, so a network with one hidden layer of $k+1$ neurons represents it exactly. This note builds the same function the other way round, as a network with k hidden layers, each of which creates one kink. One neuron per layer cannot do it (Section 5), but two can: a *hinge neuron*, which makes the kink of its layer, and a *running-sum neuron*, which adds up the kinks made so far. The hinge neurons form one chain and the running-sum neurons another, so we call such a network a *hinge-chain network*. We give the formula of every neuron (Section 2), prove that the network computes f (Section 3), and work through an example with three kinks (Section 4). You only need to know what a ReLU neuron is.

§1 The function and its hinge form

Throughout, $\mathrm{ReLU}(z) = \max\{z, 0\}$. Let $f : [a, b] \to \mathbb{R}$ be continuous and piecewise linear, with $k \geq 1$ kinks, the points where the slope changes:

$$a < t_1 < t_2 < \cdots < t_k < b.$$

Put $t_0 = a$ and $t_{k+1} = b$, let s_i be the slope of f on $[t_i, t_{i+1}]$ for $i = 0, \dots, k$, and set

$$\Delta_0 = s_0, \qquad \Delta_i = s_i - s_{i-1} \ \ (i = 1, \dots, k), \qquad d_\ell = t_\ell - t_{\ell-1} > 0 \ \ (\ell = 1, \dots, k). \tag{1}$$

So Δ_i is the change of slope at the ith kink, and d_ℓ is the gap between consecutive kinks ($d_1 = t_1 - a$). The network below is built from $f(a)$ and these numbers alone.

The starting point is the hinge form of f,

$$f(x) = f(a) + s_0\,(x - a) + \sum_{i=1}^{k} \Delta_i \ \mathrm{ReLU}(x - t_i), \qquad x \in [a, b]. \tag{2}$$

Both sides are continuous and piecewise linear, they agree at $x = a$ (every hinge vanishes there), and on $[t_i, t_{i+1}]$ both have the slope $s_0 + \Delta_1 + \cdots + \Delta_i = s_i$, so they agree everywhere. Since $\mathrm{ReLU}(x - a) = x - a$ on $[a, b]$, the linear term is just a hinge number 0, with $\Delta_0 = s_0$ and $t_0 = a$. The partial sums

$$F_\ell(x) = f(a) + \sum_{i=0}^{\ell} \Delta_i \ \mathrm{ReLU}(x - t_i), \qquad \ell = -1, 0, 1, \dots, k, \tag{3}$$

keep the first ℓ kinks of f and drop the rest ($F_{-1} = f(a)$ is the empty sum). They satisfy $F_\ell = F_{\ell-1} + \Delta_\ell \ \mathrm{ReLU}(x - t_\ell)$, and $F_k = f$ by (2).

§2 The network, neuron by neuron

The network has one input x, k hidden layers of two ReLU neurons each, and one linear output y. Layer ℓ ($\ell = 1, \dots, k$) contains

- a *hinge neuron* h_ℓ, whose kink will sit at t_ℓ, and
- a *running-sum neuron* g_ℓ.

To state the rules in one go, put $h_0 = x - a$ and $g_0 = C + f(a)$. These are not neurons (h_0 is the shifted input, g_0 a constant), but with them layer 1 obeys the same two rules as every other layer. The constant $C \geq 0$ keeps the running sum nonnegative; it is explained after Proposition 1, and $C = 0$ in the example.

For $\ell = 1, \dots, k$,

$$h_\ell = \mathrm{ReLU}\big(h_{\ell-1} - d_\ell\big), \tag{4}$$

$$g_\ell = \mathrm{ReLU}\big(g_{\ell-1} + \Delta_{\ell-1}\, h_{\ell-1}\big), \tag{5}$$

and the output is

$$y = \Delta_k\, h_k + g_k - C. \tag{6}$$

The hinge neuron only shifts the previous hinge by the gap d_ℓ. The running-sum neuron passes its own value on and adds the previous hinge, weighted by the change of slope that hinge stands for. Table 1 lists every neuron with its input; the first layer, written out, reads $h_1 = \mathrm{ReLU}(x - t_1)$ and $g_1 = \mathrm{ReLU}\big(s_0 x + f(a) - s_0 a + C\big)$.

Neuron	Input z	Output	Equals (Proposition 1)
h_1	$x - t_1$	$\mathrm{ReLU}(z)$	$\mathrm{ReLU}(x - t_1)$
g_1	$s_0\, x + f(a) - s_0\, a + C$	$\mathrm{ReLU}(z)$	$C + F_0(x)$
h_ℓ, $\ell \geq 2$	$h_{\ell-1} - d_\ell$	$\mathrm{ReLU}(z)$	$\mathrm{ReLU}(x - t_\ell)$
g_ℓ, $\ell \geq 2$	$g_{\ell-1} + \Delta_{\ell-1}\, h_{\ell-1}$	$\mathrm{ReLU}(z)$	$C + F_{\ell-1}(x)$
y	$\Delta_k\, h_k + g_k - C$	z	$f(x)$

Table 1: Every neuron of the network. The last column is what the neuron turns out to compute (Proposition 1); F_ℓ is the partial sum (3). Hinge neurons are in red, running-sum neurons in green.

In matrix form, with the ReLU applied entrywise, layer 1 is

$$\begin{pmatrix} h_1 \\ g_1 \end{pmatrix} = \mathrm{ReLU}\left(\begin{pmatrix} 1 \\ s_0 \end{pmatrix} x + \begin{pmatrix} -t_1 \\ f(a) - s_0\, a + C \end{pmatrix} \right),$$

layer $\ell = 2, \dots, k$ is

$$\begin{pmatrix} h_\ell \\ g_\ell \end{pmatrix} = \mathrm{ReLU}\left(\begin{pmatrix} 1 & 0 \\ \Delta_{\ell-1} & 1 \end{pmatrix} \begin{pmatrix} h_{\ell-1} \\ g_{\ell-1} \end{pmatrix} + \begin{pmatrix} -d_\ell \\ 0 \end{pmatrix} \right),$$

and the output is

$$y = \begin{pmatrix} \Delta_k & 1 \end{pmatrix} \begin{pmatrix} h_k \\ g_k \end{pmatrix} - C.$$

In the 2×2 matrix, the 1 in the upper left says that a hinge neuron reads the previous hinge, and the 0 in the upper right that it never reads the running sum. The 1 in the lower right says that the running-sum neuron passes its own value on, and the entry $\Delta_{\ell-1}$ in the lower left that it adds the previous hinge.

§3 What every neuron computes

The hinge neurons never need x after the first layer, because a shifted hinge is a hinge of a hinge.

Lemma 1. *For every $u \in \mathbb{R}$ and every $d \geq 0$,* $\mathrm{ReLU}\big(\mathrm{ReLU}(u) - d\big) = \mathrm{ReLU}(u - d)$.

Proof. If $u \geq 0$, then $\mathrm{ReLU}(u) = u$ and the two sides are equal. If $u < 0$, then $\mathrm{ReLU}(u) - d = -d \leq 0$ and $u - d < 0$, so both sides are 0. □

Proposition 1 (What every neuron computes). *Let $C \geq 0$ be such that*

$$C + F_i(x) \geq 0 \qquad \text{for all } x \in [a, b] \text{ and } i = 0, 1, \dots, k-1. \tag{7}$$

Then, for every $x \in [a, b]$,

(a) $h_\ell(x) = \mathrm{ReLU}(x - t_\ell)$ *for* $\ell = 0, 1, \dots, k$;
(b) $g_\ell(x) = C + F_{\ell-1}(x)$ *for* $\ell = 0, 1, \dots, k$;
(c) $y(x) = f(x)$;
(d) for $\ell = 1, \dots, k$: $F_{\ell-1}(x) = f(x)$ *if* $x \leq t_\ell$, *and* $F_{\ell-1}(x) = f(t_\ell) + s_{\ell-1}(x - t_\ell)$ *if* $x \geq t_\ell$.

In words: h_ℓ is a hinge with its kink at t_ℓ (a). The running sum $g_\ell - C$ is f itself up to the ℓth kink and goes on as a straight line, with the slope $s_{\ell-1}$ of the last piece it has seen, from there on (d); it therefore has exactly the first $\ell - 1$ kinks of f (b). The output adds the last hinge (c).

Proof. (a) For $\ell = 0$, $h_0 = x - a = \mathrm{ReLU}(x - t_0)$ because $x \geq a$. If $h_{\ell-1} = \mathrm{ReLU}(x - t_{\ell-1})$, then (4) and Lemma 1 with $d = d_\ell \geq 0$ give

$$h_\ell = \mathrm{ReLU}\big(\mathrm{ReLU}(x - t_{\ell-1}) - d_\ell\big) = \mathrm{ReLU}(x - t_{\ell-1} - d_\ell) = \mathrm{ReLU}(x - t_\ell).$$

(b) For $\ell = 0$, $g_0 = C + f(a) = C + F_{-1}$. Let $\ell \geq 1$ and assume $g_{\ell-1} = C + F_{\ell-2}$. By (a), the input of g_ℓ is

$$g_{\ell-1} + \Delta_{\ell-1}\, h_{\ell-1} = C + F_{\ell-2} + \Delta_{\ell-1}\, \mathrm{ReLU}(x - t_{\ell-1}) = C + F_{\ell-1}(x),$$

which is ≥ 0 by (7). The ReLU in (5) therefore changes nothing, and $g_\ell = C + F_{\ell-1}$.

(c) By (6), (a) and (b), $y = \Delta_k\, \mathrm{ReLU}(x - t_k) + F_{k-1}(x) = F_k(x) = f(x)$.

(d) If $x \leq t_\ell$, then $\mathrm{ReLU}(x - t_j) = 0$ for all $j \geq \ell$, so $f(x) = F_k(x) = F_{\ell-1}(x)$. On $[t_{\ell-1}, \infty)$ the hinges $\mathrm{ReLU}(x - t_i)$ with $i \leq \ell - 1$ are all linear, so $F_{\ell-1}$ is affine there, with slope $\Delta_0 + \cdots + \Delta_{\ell-1} = s_{\ell-1}$; by the first part it equals $f(t_\ell)$ at t_ℓ. □

The constant C. Condition (7) says that the input of every running-sum neuron is nonnegative, so that its ReLU does nothing and the neuron really passes the sum on. (A hinge neuron is the opposite: it is there to clip.) If some F_i dips below zero on $[a, b]$, a constant C lifts the whole sum, and the output (6) takes it off again. The smallest admissible constant is

$$C_* = \max\Big\{0,\ -\min_{0 \leq i \leq k-1}\ \min_{x \in [a,b]} F_i(x)\Big\},$$

where the inner minimum is attained at a, at b, or at a kink, because F_i is piecewise linear. A choice that needs no computation is

$$C = \max\Big\{0,\ \sum_{i=0}^{k-1} |\Delta_i|\,(b - t_i) - f(a)\Big\},$$

because $0 \leq \mathrm{ReLU}(x - t_i) \leq b - t_i$ on $[a, b]$, so that $F_j(x) \geq f(a) - \sum_{i \leq j} |\Delta_i|(b - t_i)$ for every $j \leq k - 1$. It is usually far too large; in the example below it is 21, whereas $C_* = 0$.

Condition (7) cannot be dropped. Fix x, write $u_\ell = C + F_{\ell-1}(x)$ for the value that g_ℓ should have, and $e_\ell = g_\ell - u_\ell$ for its error, so that $e_0 = 0$. The hinge neurons do not depend on the running-sum neurons, so (a) holds regardless, and the input of g_ℓ is $u_\ell + e_{\ell-1}$. Since $\mathrm{ReLU}(v) \geq v$, we get $e_\ell \geq e_{\ell-1}$, and if $u_\ell + e_{\ell-1} < 0$ then $e_\ell = -u_\ell > e_{\ell-1}$. So the error never decreases, and it becomes positive at the first ℓ with $C + F_{\ell-1}(x) < 0$. As $y = f + e_k$, we obtain $y(x) > f(x)$ whenever (7) fails at x.

§4 A worked example with three kinks

Take $[a, b] = [0, 4]$ and the function f through the points $(0, 2), (1, 4), (2, 3), (3, 5), (4, 2)$. It has $k = 3$ kinks, at $t = (1, 2, 3)$, and

$$f(a) = 2, \qquad s = (2, -1, 2, -3), \qquad (\Delta_0, \Delta_1, \Delta_2, \Delta_3) = (2, -3, 3, -5), \qquad d = (1, 1, 1).$$

The partial sums are $F_0(x) = 2 + 2x \geq 2$, $F_1(x) = 2 + 2x - 3\,\mathrm{ReLU}(x - 1) \geq 1$ and $F_2(x) = F_1(x) + 3\,\mathrm{ReLU}(x - 2) \geq 2$ on $[0, 4]$, so (7) holds with $C = 0$. The rules of Section 2 become

$$\begin{aligned} h_1 &= \mathrm{ReLU}(x - 1), & g_1 &= \mathrm{ReLU}(2x + 2), \\ h_2 &= \mathrm{ReLU}(h_1 - 1), & g_2 &= \mathrm{ReLU}(g_1 - 3\,h_1), \\ h_3 &= \mathrm{ReLU}(h_2 - 1), & g_3 &= \mathrm{ReLU}(g_2 + 3\,h_2), \end{aligned}$$

$$y = g_3 - 5\,h_3,$$

and Table 2 gives the same network as weights and biases.

Layer	Input	Weights	Bias
1	x	$\begin{pmatrix} 1 \\ 2 \end{pmatrix}$	$\begin{pmatrix} -1 \\ 2 \end{pmatrix}$
2	(h_1, g_1)	$\begin{pmatrix} 1 & 0 \\ -3 & 1 \end{pmatrix}$	$\begin{pmatrix} -1 \\ 0 \end{pmatrix}$
3	(h_2, g_2)	$\begin{pmatrix} 1 & 0 \\ 3 & 1 \end{pmatrix}$	$\begin{pmatrix} -1 \\ 0 \end{pmatrix}$
output	(h_3, g_3)	$\begin{pmatrix} -5 & 1 \end{pmatrix}$	0

Table 2: The example as weights and biases. Layers 1 to 3 are followed by a ReLU, the output is not.

By Proposition 1, the neurons compute

$$h_\ell(x) = \mathrm{ReLU}(x - \ell) \quad (\ell = 1, 2, 3), \qquad g_1(x) = 2x + 2,$$

$$g_2(x) = 2x + 2 - 3\,\mathrm{ReLU}(x - 1) = \begin{cases} 2x + 2 & 0 \leq x \leq 1, \\ 5 - x & 1 \leq x \leq 4, \end{cases}$$

$$g_3(x) = 2x + 2 - 3\,\mathrm{ReLU}(x - 1) + 3\,\mathrm{ReLU}(x - 2) = \begin{cases} 2x + 2 & 0 \leq x \leq 1, \\ 5 - x & 1 \leq x \leq 2, \\ 2x - 1 & 2 \leq x \leq 4, \end{cases}$$

$$y(x) = g_3(x) - 5\,\mathrm{ReLU}(x - 3) = f(x).$$

As (d) predicts, g_2 is f up to $t_2 = 2$ and then goes on straight with the slope -1 that f has on $[1, 2]$, and g_3 is f up to $t_3 = 3$ and then goes on with the slope 2 of $[2, 3]$. The output subtracts the last hinge $5\,\mathrm{ReLU}(x - 3)$, which bends the line down to the slope -3 of the last piece. Table 3 lists the values of every neuron at the points $x = 0, \ldots, 4$.

x	0	1	2	3	4
h_1	0	0	1	2	3
g_1	2	4	6	8	10
h_2	0	0	0	1	2
g_2	2	4	3	2	1
h_3	0	0	0	0	1
g_3	2	4	3	5	7
y	2	4	3	5	2
f	2	4	3	5	2

Table 3: The value of every neuron of the example at the points $x = 0, \dots, 4$, and the target f.

§5 Remarks

One neuron per layer is not enough. A layer with a single ReLU neuron is the map $u \mapsto \mathrm{ReLU}(wu+c)$, which is monotone: non-decreasing if $w \geq 0$, non-increasing if $w \leq 0$. A composition of monotone maps is monotone, and so is the final affine map. Hence a ReLU network with one neuron per hidden layer is a monotone function of x, and it cannot represent any f with an interior local extremum, such as the example of Section 4, which peaks at $x = 1$. (One can check that such a network is in fact a single ramp: constant, one sloped piece, constant.) So two neurons per layer is the least that works for every f, and the construction above shows that it suffices.[1]

Cost. The deep network has $2k$ ReLU neurons, against $k + 1$ for the shallow one of the introduction, so depth saves nothing here; it only makes the kinks appear one layer at a time.

Only on $[a, b]$. Condition (7) and the identities of Proposition 1 are claimed for $x \in [a, b]$ only. Outside the interval nothing is claimed: a running-sum neuron may start to clip there, and y need not continue f.

With skip connections, one hidden neuron per layer suffices. Let a residual stream carry the pair $\big(x, F_{\ell-1}(x)\big)$ and let block ℓ add one hinge,

$$\begin{pmatrix} x \\ F_{\ell-1} \end{pmatrix} \longmapsto \begin{pmatrix} x \\ F_{\ell-1} \end{pmatrix} + \begin{pmatrix} 0 \\ \Delta_\ell \end{pmatrix} \mathrm{ReLU}(x - t_\ell) = \begin{pmatrix} x \\ F_\ell \end{pmatrix}.$$

Each block has a single hidden neuron $\mathrm{ReLU}(x - t_\ell)$, and no constant C is needed, since the running sum never passes through a ReLU. The map from the input to the starting pair $\big(x, F_0(x)\big)$ is affine, and after block k the second coordinate is $F_k = f$.

[1] Hanin and Sellke, *Approximating continuous functions by ReLU nets of minimal width* (2017), show that, for ReLU networks of any depth, width $d + 1$ is the minimum for approximating every continuous function of d variables. For $d = 1$ this is the width two used here.